Fall 2025

Discovering New Exploit Techniques Through Generalizing Known Exploits

Dissertation Defense

Yihui (Kyle) Zeng

Tiffany Bao (Chair)

Adam Doupé

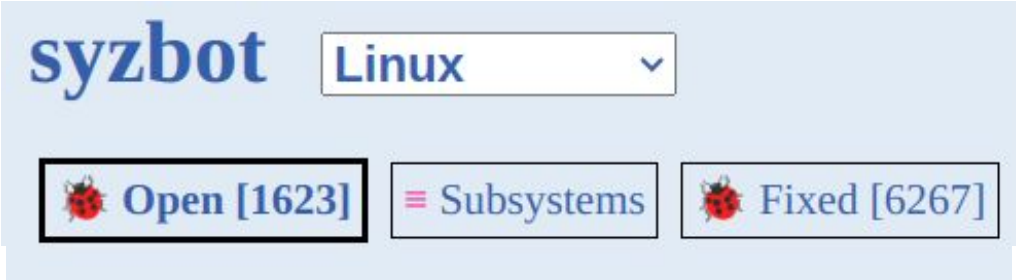
David Brumley

Ruoyu Wang

Yan Shoshitaishvili

Software Programs Are Buggy

Linux: Billions of users



Plenty Of Exploits

Exploits: program that abuses vulnerabilities to achieve unauthorized access

Exploit Reward Program

Exploits Are Understudied

- Each exploit reveals a full chain of weaknesses (abused by techniques)

- Can we learn anything from known exploits?

Generalizing known exploits can lead to the discovery of new exploitation techniques

Agenda

- Background
- Exploit Generalization
- Case 1: Playing For K(H)eaps
- Case 2: RetSpill
- Case 3: ropbot
- Conclusion

Exploit Generalization

1. Understand techniques
2. Identify dominating factors
3. Derive new techniques

Step 1: Understand Techniques

- Even exploit developers may not understand the techniques

Google P0:

Prefetch does not work

against KPTI

willsroot:

Hmm. Not true.

=> EntryBleed

* Exploiting CVE-2022-42703 - Bringing back the stack attack

* EntryBleed: Breaking KASLR under KPTI with Prefetch (CVE-2022-4543)

EntryBleed – The Technique

- Step 1: warm up `entry_SYSCALL_64` using syscalls
- Step 2: prefetch a candidate address and observe its timing, if it returns fast, it is `entry_SYSCALL_64`

Step 2: Identify Dominating Factors

- Factors/system behaviors dominating the feasibility of techniques
- Vary in different domains

EntryBleed – Dominating Factor

- Summary: abuse CPU caching mechanisms to infer the existence of pages and use page existence to infer the kernel base
- Dominating factor: page existence

Step 3: Derive New Techniques

- New approach to interacting with the dominating factor

  => new technique

EntryBleed-Inspired Technique

- Other ways to infer the existence of pages?
- Step 1: prefetch any address once to load it into TLB
- Step 2: prefetch it again
- Step 3: timing tells the existence

Limitations

- Can only discover techniques interacting with known dominating factors
- Doesn’t necessarily scale to other domains

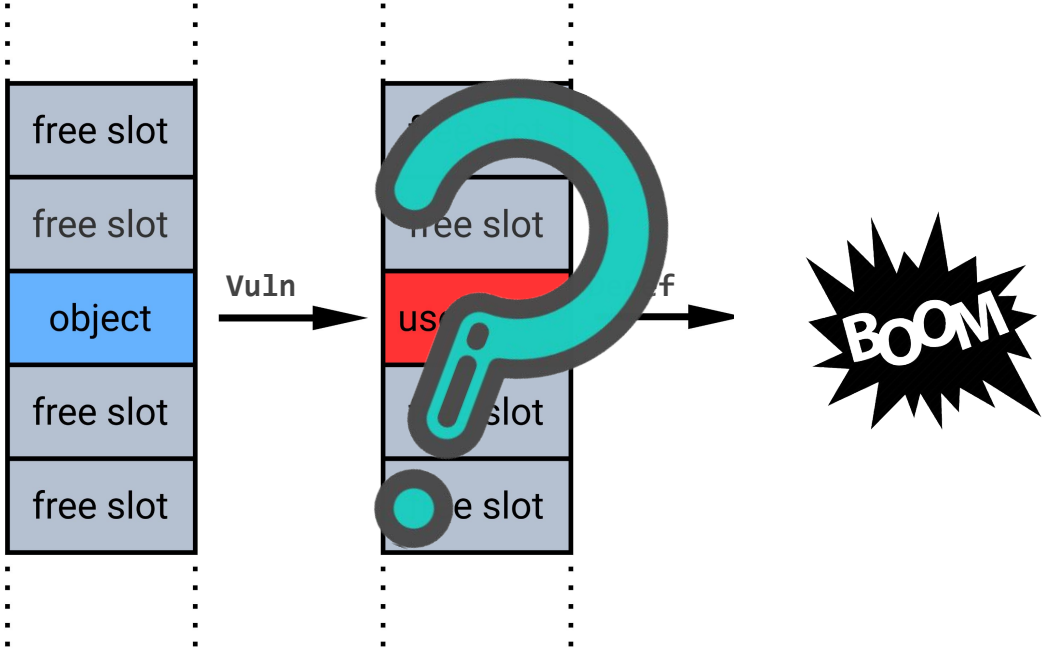
Agenda

- Background
- Exploit Generalization
- Case 1: Playing For K(H)eaps
- Case 2: RetSpill
- Case 3: ropbot
- Conclusion

Playing for K(H)eaps:
Understanding and Improving Linux Kernel Exploit Reliability

- Generalize kernel exploit stabilization techniques
- Discover four dominating factors
- Derive new stabilization techniques

Linux Kernel Heap Exploit



Linux kernel exploits are dangerous

Linux kernel exploits are known to be unreliable

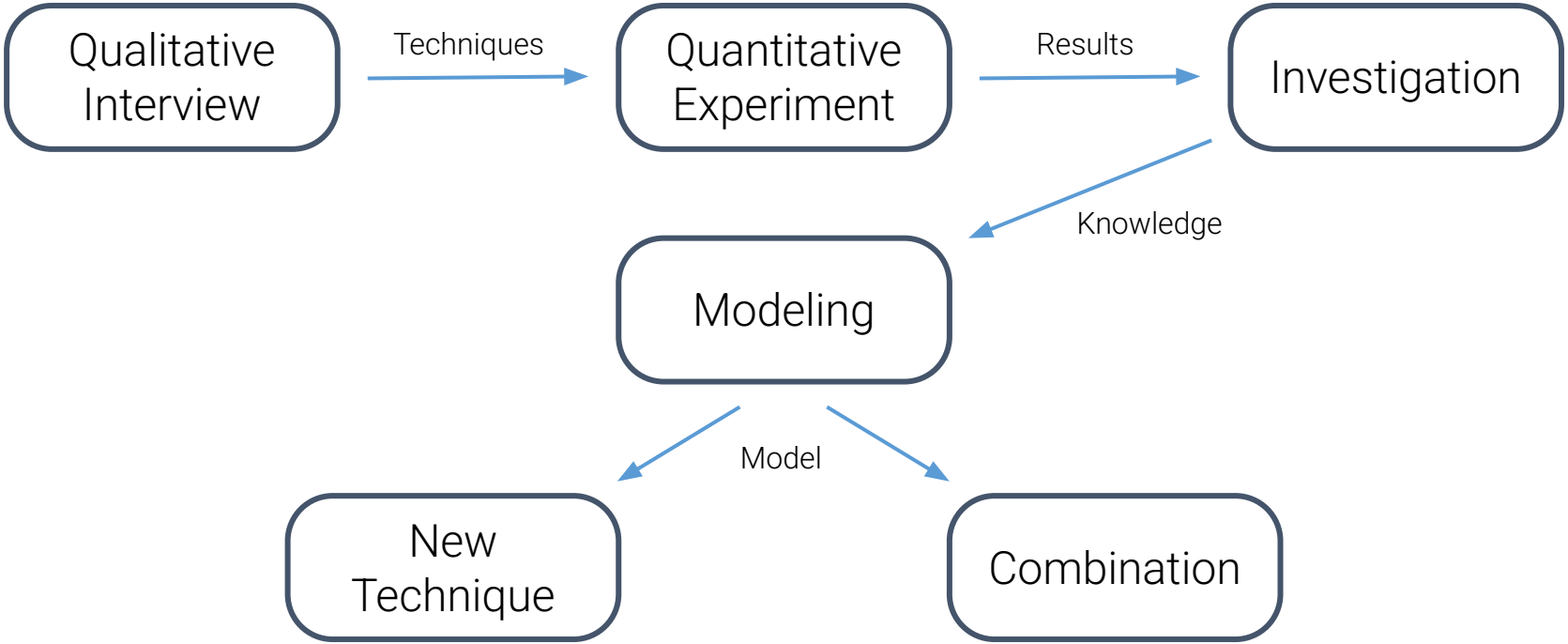
Exploit stabilization heavily relies on personal expertise

Systematically study why Linux kernel heap-based exploits are unreliable

Research Questions

- What are the commonly used exploit stabilization techniques?
- How effective are existing techniques?
- Why do existing techniques work?
- Can we generalize them to discover new techniques?

Our Approach



Technique Collection

11 Linux kernel security experts

Defragmentation

Heap Grooming

Single-Thread Heap Spray

Multi-Process Heap Spray

CPU Pinning

*obtained exemption from IRB

Quantitative Experiment

- Real-world exploits: 17 public exploits for distinct CVEs
- Baseline exploits: strip away existing techniques
- Exploit variants: apply one single technique to baseline

85 samples in total

Quantitative Experiment Result

	Baseline	Defragment	Heap Grooming	Single-Thread Spray	Multi-Process Spray	CPU Pinning
Success	38.61%	31.88%	74.40%	61.83%	82.55%	51.51%

Evaluation result of all techniques

Quantitative Experiment Result – Cont.

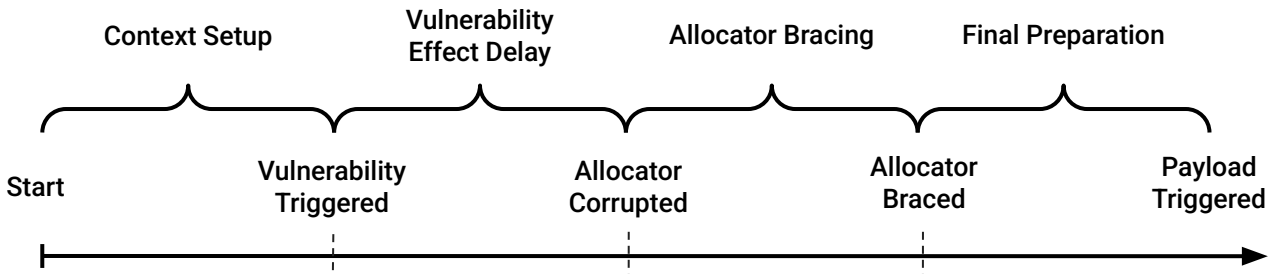
	Baseline	Defragmentation
Success	13.05%	42.64%

Evaluation Result for OOB Exploits

	Baseline	Defragmentation
Success	49.26%	27.40%

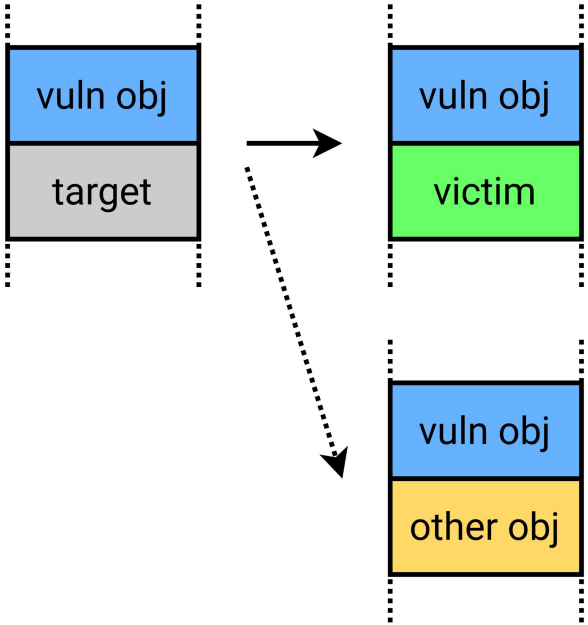
Evaluation Result for non-OOB Exploits

Kernel Heap Exploit Model



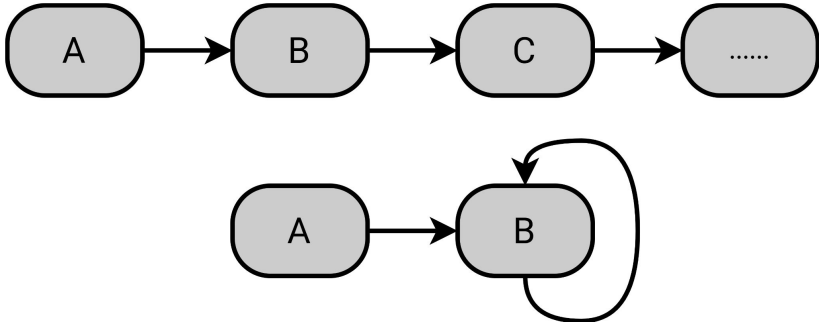
Critical Phases

Slot-Critical Phase



OOB Exploits

Allocator-Critical Phase

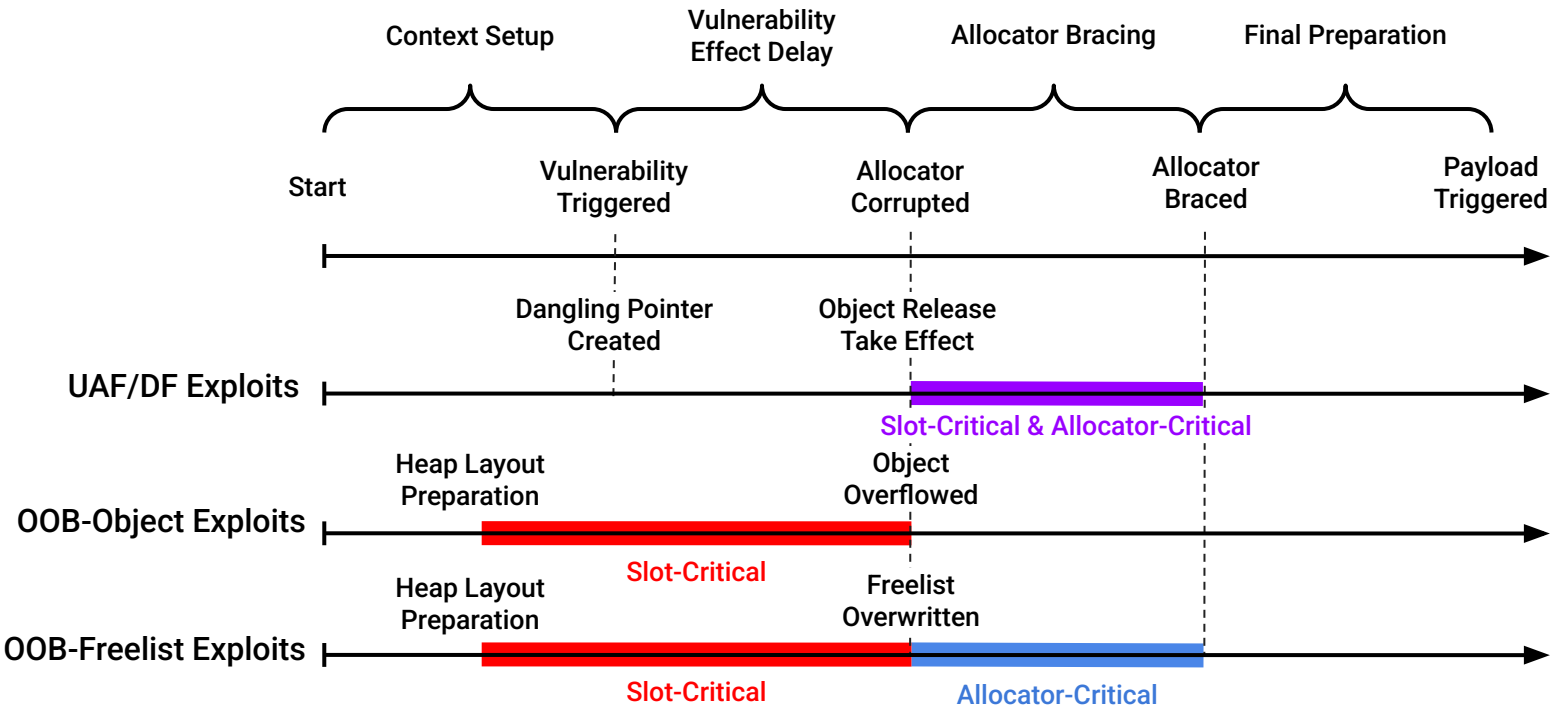


DF Exploits

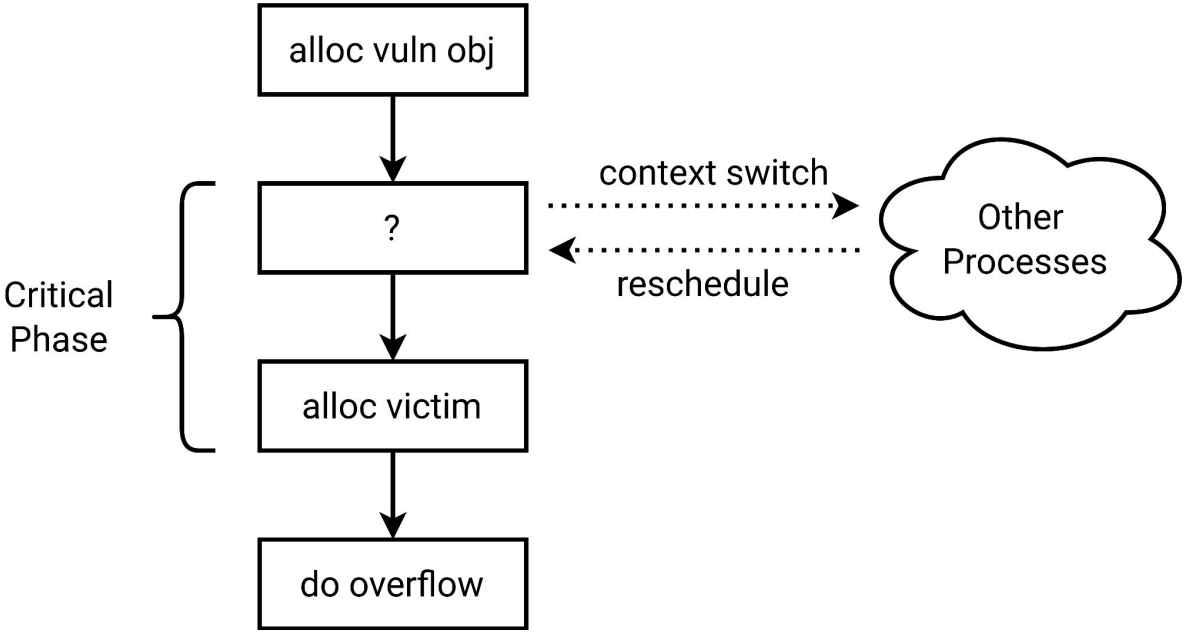
Dominating Factors

- Unknown Heap Layout
- Unexpected Heap Usage
- Unwanted Task Migration
- Unpredictable Corruption Timing

Kernel Heap Exploit Model



Context Conservation



Context Conservation – Cont.

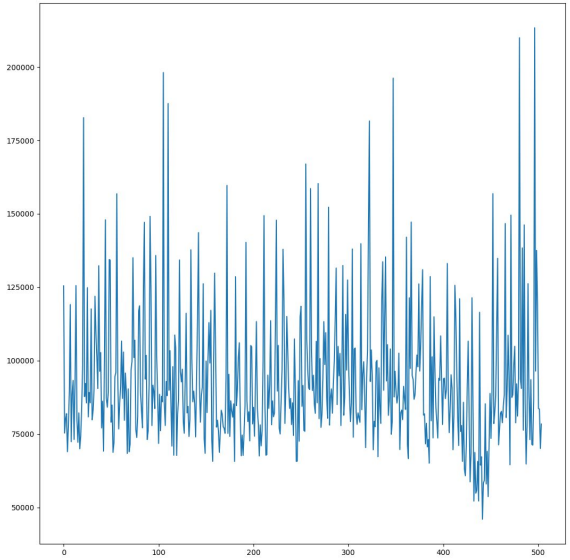
Use Time Stamp Counter (TSC) as the context-switch indicator

```
tsc1 = rdtsc()
tsc2 = rdtsc()
diff = tsc2 - tsc1
```

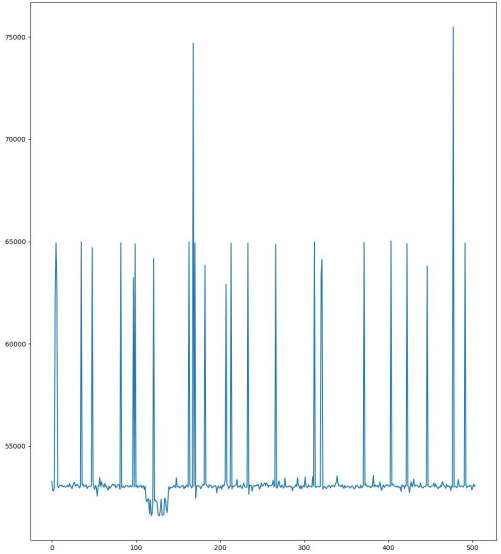
If diff is huge, then it is a fresh time slice

	Baseline	Context Conservation
Idle	62.48%	64.07%
Busy	36.75%	49.84%

Context Conservation – Prefetch Attack

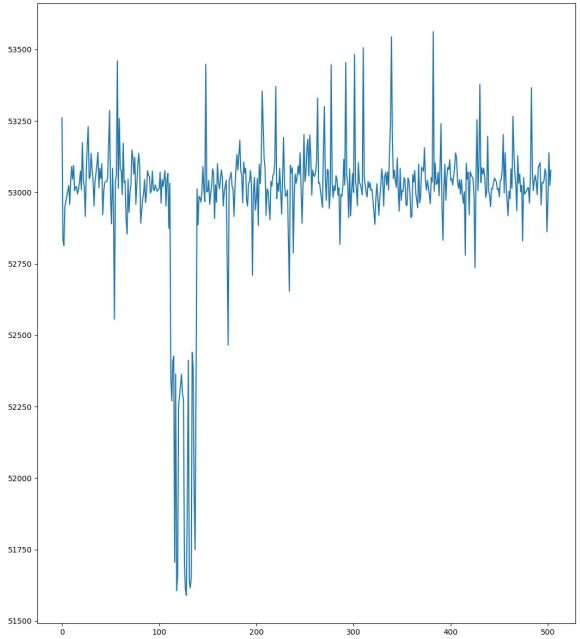


Prefetch Attack in VM



Prefetch Attack in Physical Machine

Context Conservation – Prefetch Attack



Prefetch Attack with Context Conservation

100% success rate
in both VM and physical machine

Combo Technique

- Unknown Heap Layout ← Defragmentation
- Unexpected Heap Usage ← Context Conservation
- Unwanted Task Migration ← CPU Pinning
- Unpredictable Corruption Timing ← Multi-Process Heap Spray

What if we combine them?

Combo Technique – Cont.

Exploit Variant: baseline + applicable techniques

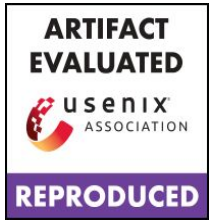
	Baseline	Real-world	Combo
Success	36.51%	66.99%	91.15%

Evaluation Result

K(H)eaps Takeaways

- Analyzed and generalized known techniques and formulated a model
- Discovered a new technique that improve exploit reliability by 14.87%
- Designed a technique combination that improves exploit reliability by 135.53%

https://github.com/sefcom/KHeaps



Agenda

- Background
- Exploit Generalization
- Case 1: Playing For K(H)eaps
- Case 2: RetSpill
- Case 3: ropbot
- Conclusion

RetSpill: Igniting User-Controlled Data to Burn Away Linux Kernel Protections

- Generalize control-flow hijacking exploits in Linux
- Identify user data access as the dominating factor
- Discover the a new attack surface, leading ot the RetSpill exploitation technique

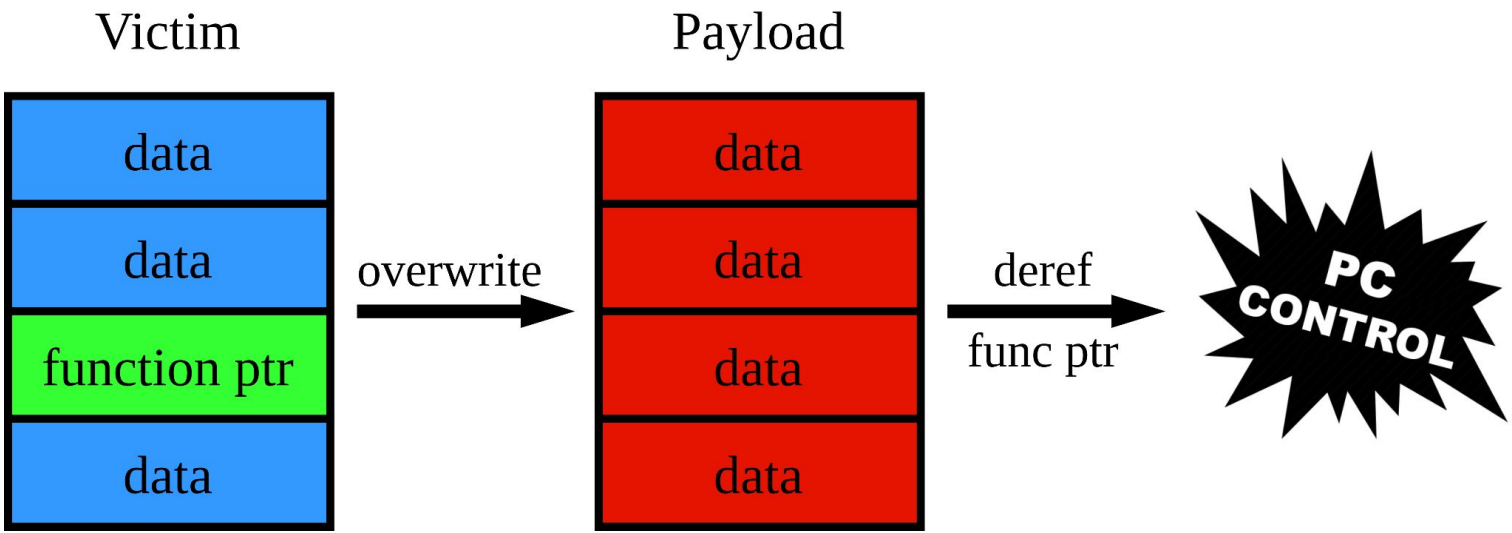
Linux Kernel Security

Google launched kernelCTF program to collect Linux kernel exploits

52 out of 70 are heap-based control-flow hijacking exploits*

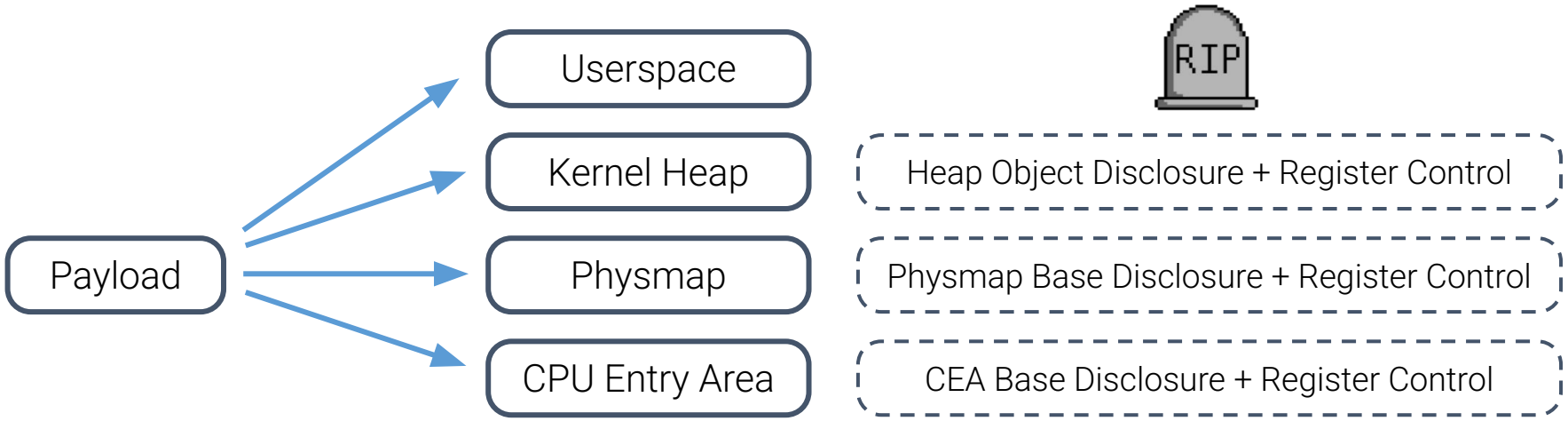
* https://github.com/google/security-research/tree/master/pocs/linux/kernelctf

Linux Kernel Heap Exploit

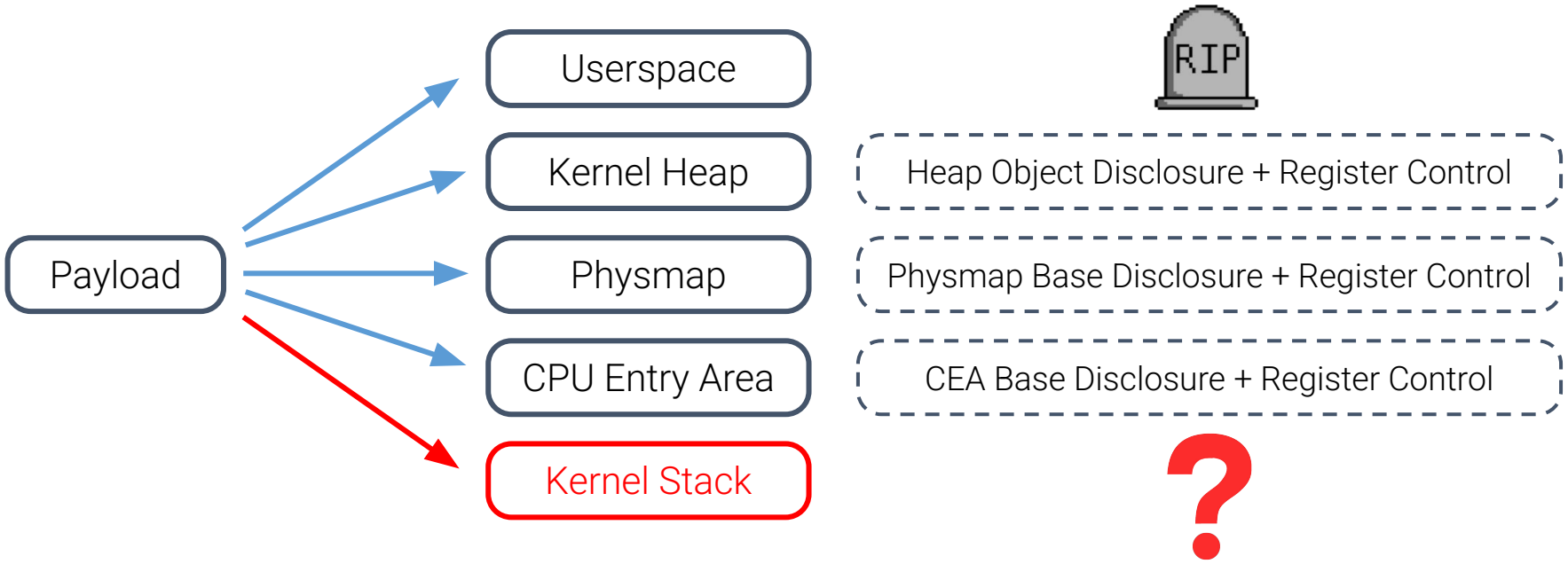


* According to Kernel Exploit Recipes Notebook by Aug 10th, 2022

PC-control to LPE: User Data Access



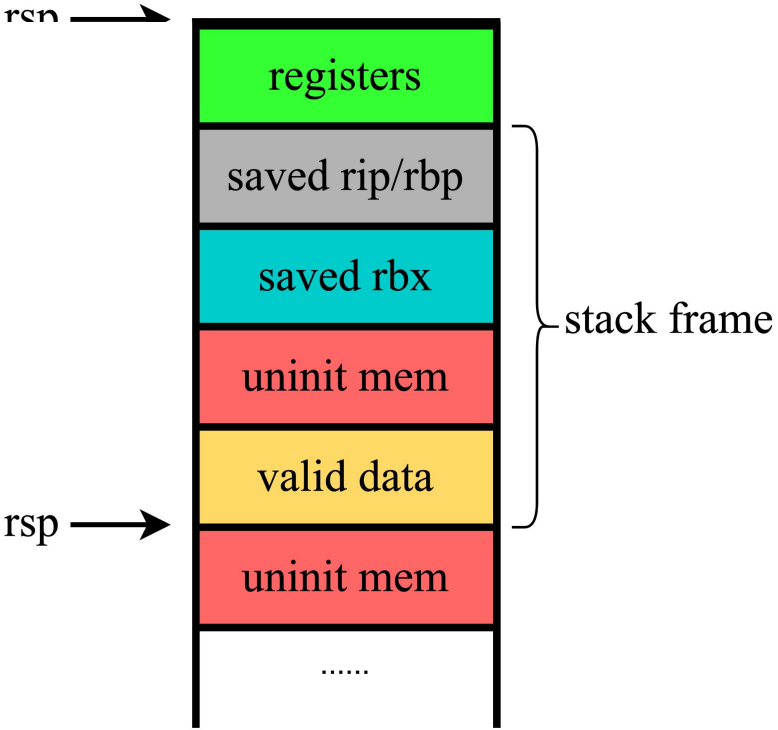
New Attack Surface



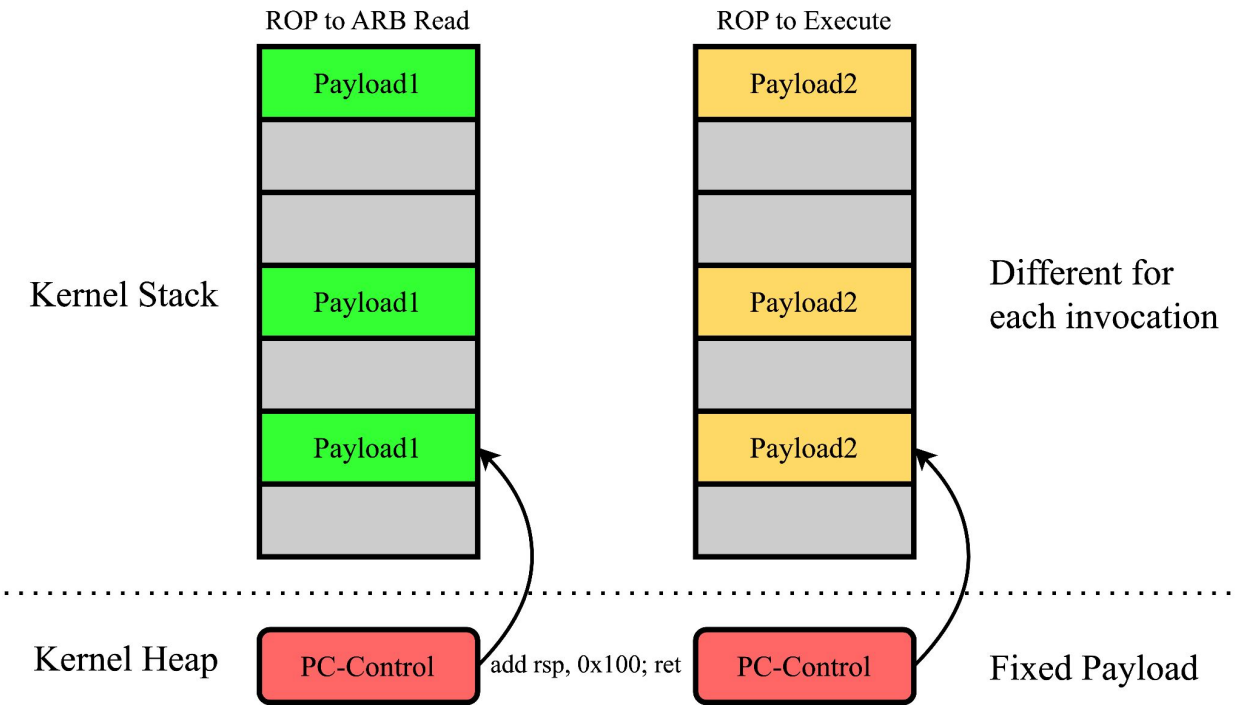
Systematically study the impact of on-stack user data on kernel security

Data Spillage Source

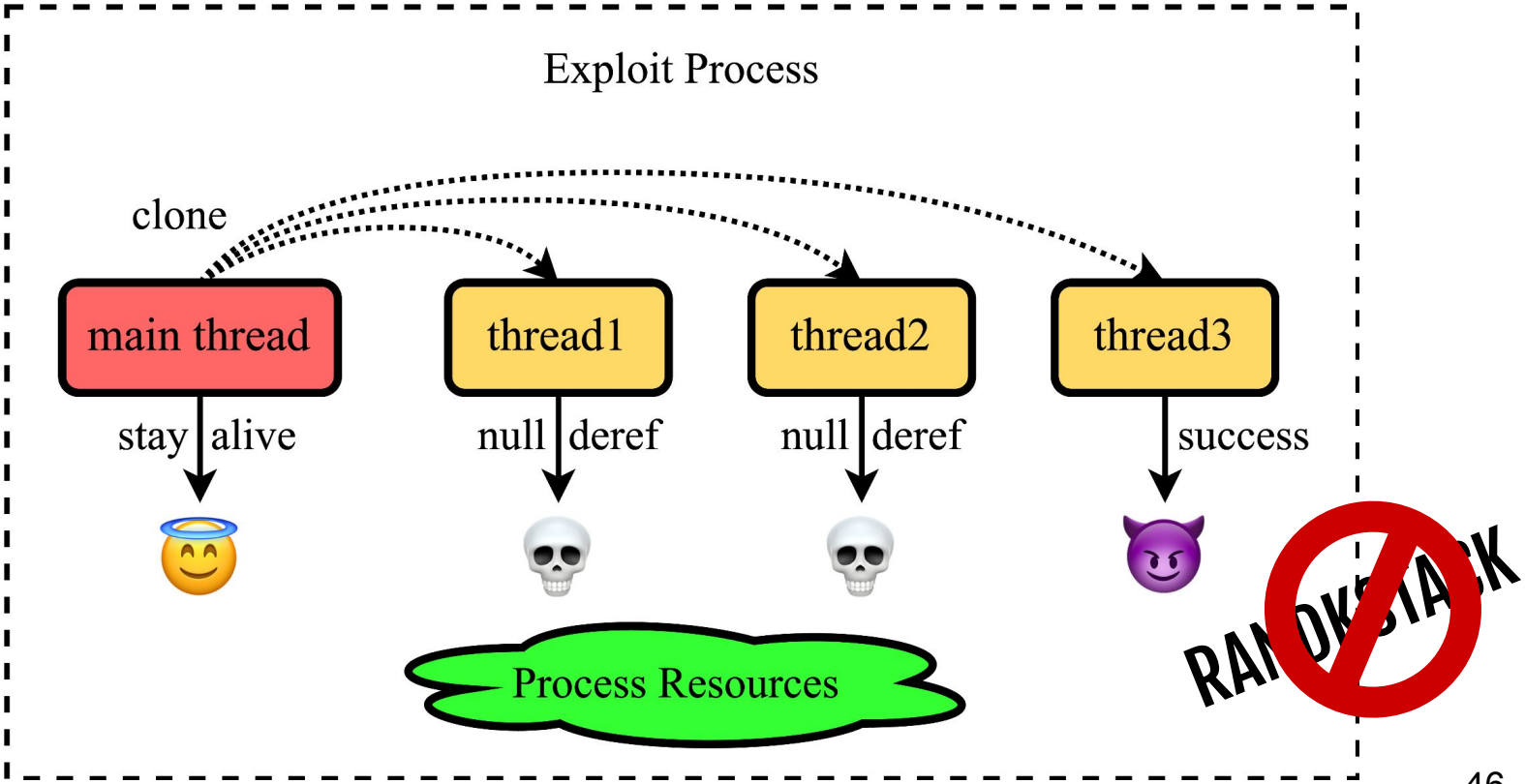
- Preserved Registers
- Calling Convention
- Valid Data
- Uninitialized Memory



Primitive 1: Rewritable Payload



Primitive 2: Crash-Resilient ROP

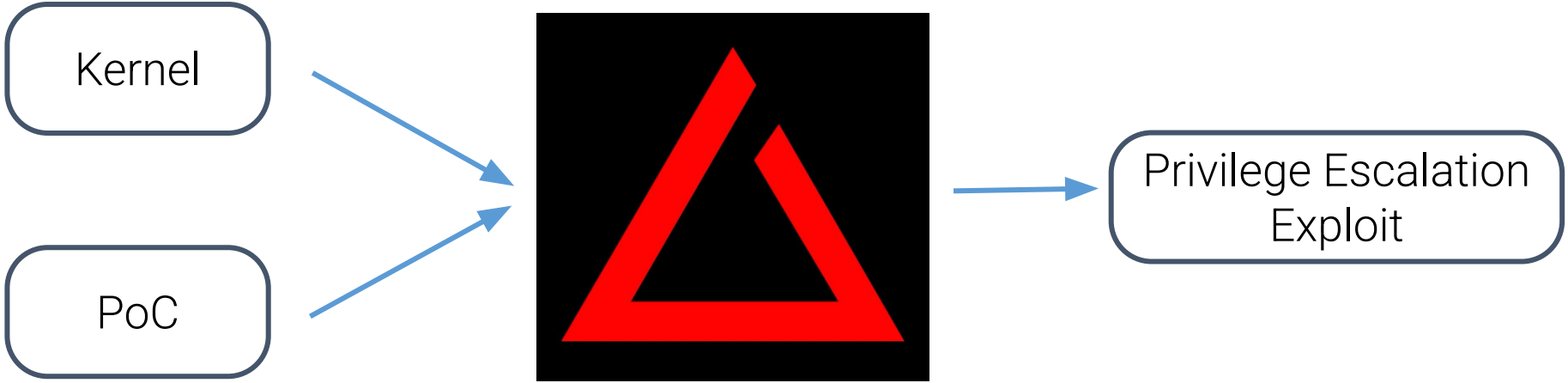


Break User/Kernel Boundary

- Rewritable Payload
  - turn one PC-Control into many without reliability degradation
- Crash-Resilient ROP
  - enhanced resiliency

RetSpill: Reliable unlimited arbitrary read/write/exec given one PC-Control

IGNI: Break User/Kernel Boundary Automatically

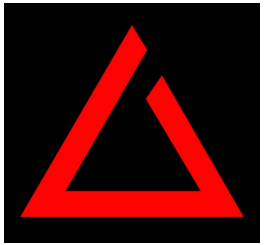


IGNI's high-level workflow

IGNI: Break User/Kernel Boundary Automatically

IGNI's high-level workflow

IGNI: Break User/Kernel Boundary Automatically



Turn ***20/22*** PoC to exploits ***automatically***

	Valid Data	Preserved Registers	Calling Convention	Uninitialized Memory	Total
Gadget	1.1	6.1	3.9	5.5	16.5

# of on-stack userspace data

RetSpill vs Mitigations

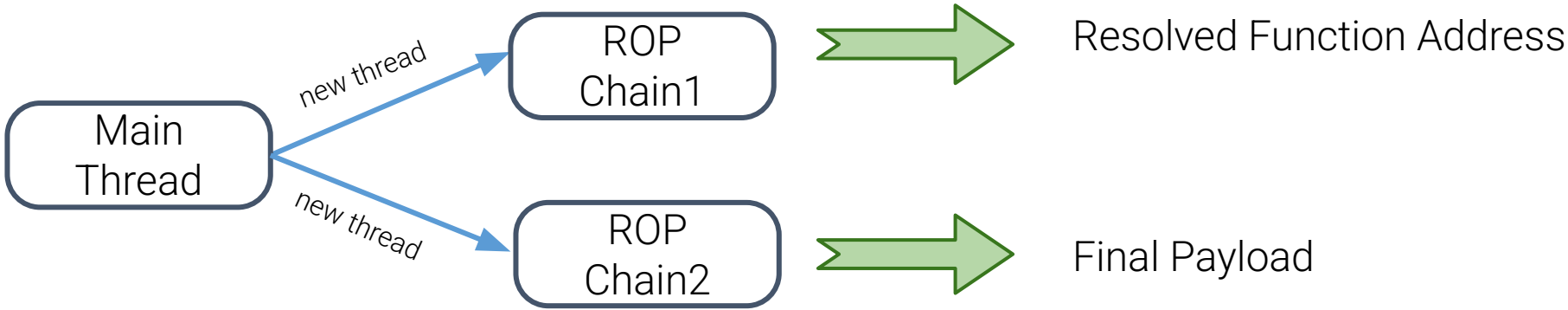
Mitigation	PC-Control Achievable?	RetSpill Works?	Deployed?
SMEP/SMAP/KPTI	✓	✓	✓
RANDKSTACK	✓	✓	✓
STACKLEAK	✓	✓	✗
FG-KASLR	✓	✓	✗
KCFI/IBT	✓	✓	✗
Shadow Stack	✓	✓ ?	✗
CFI+Shadow Stack	✗	✗	✗

Case Study: FG-KASLR Bypass

FG-KASLR: Function-Granular KASLR

Function-Granular: ROP gadgets available

Authors of FG-KASLR updated its design after our report



RetSpill Takeaways

- Generalize known CFH exploitation techniques
- Identified user data access as the dominating factor
- Discover the RetSpill exploitation technique
- Systematically study RetSpill and demonstrate its severity

Agenda

- Background
- Exploit Generalization
- Case 1: Playing For K(H)eaps
- Case 2: RetSpill
- Case 3: ropbot
- Conclusion

ropbot: Architecture-Agnostic Code Reuse Attack Synthesis

- Generalize code-reuse attack generation techniques
- Identify gadget chainability as the dominating factor
- Design an algorithm that guarantees chainability to improve performance

Existing Works

	Algorithm	Arch
SGC	BGAT	x86/x64
exrop	BGAT	x64
RiscyROP	BGAT	AArch64/RISC-V
crackers	BGAT	x64
arcanist	BGAT	x64/ARM

BGAT: bounded generate-and-test

Bounded Generate-and-Test

- Enumerate all gadget combinations (with a bound N)
  - Check whether the gadget list works
- Heuristics to list possible combinations first
- Time complexity: O(n^N)

G1: pop rax; jmp rcx;

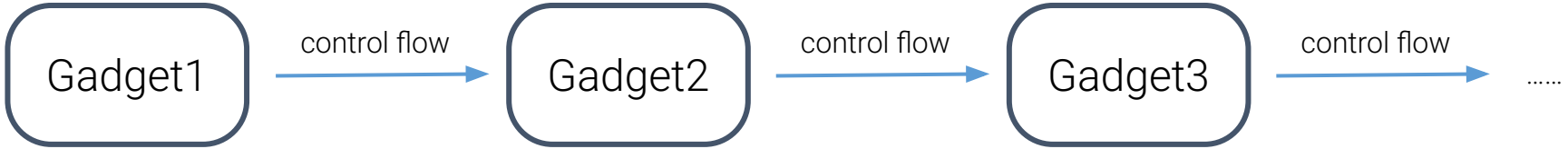
G2: pop rbx; jmp rdx

Dominating factor: Chainability

What if:

We have a special way of grouping gadgets to guarantee chainability?

Code Reuse Attack



Gadget: a unit that maintains control flow

Gadget

- Definition: code sequence that can *potentially* move a symbolic value into PC in a fully symbolic state (i.e. symbolic registers, stack, memory)

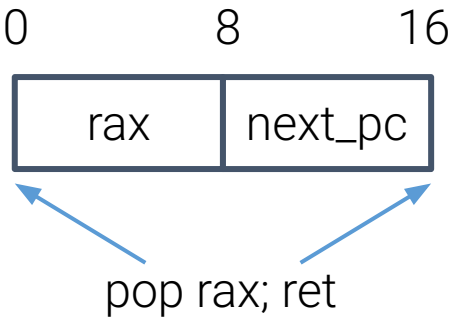
- ret ⇔ pop PC
- jmp rax ⇔ mov PC, rax

Gadget

- Advantages
  a. Architecture agnostic
  b. Accounts for conditional branches
  c. More gadgets (gadgets can span multiple basic blocks)
  d. Flexible, can find new gadgets
- Two new gadget types: GOT gadgets and “ldr pc” gadgets

Self-Contained Property

- Has positive SP change (before and after the gadget)
- Take next PC from within the SP change
- No conditional branches



ROP Block

- A gadget or a sequence of gadgets that is self-contained
- By definition, self-contained gadgets are rop blocks (e.g. pop rax; ret)
- rop block + rop block = rop block (guaranteed to be chainable)

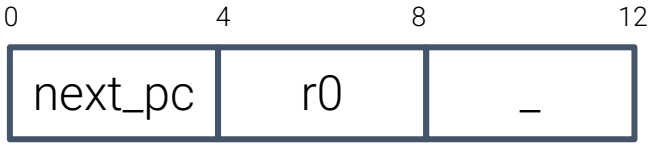
ROP Block Chaining

ldr r0, [sp, #4]; ldr lr, [sp], #4; add sp, #8; bx lr

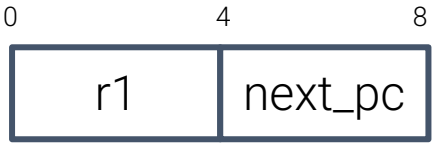
pop.w {r1, lr}; bx lr

mov eax, [esp+4]; mov ebx, [esp]; add esp, 0xc; jmp ebx

pop ebx; ret

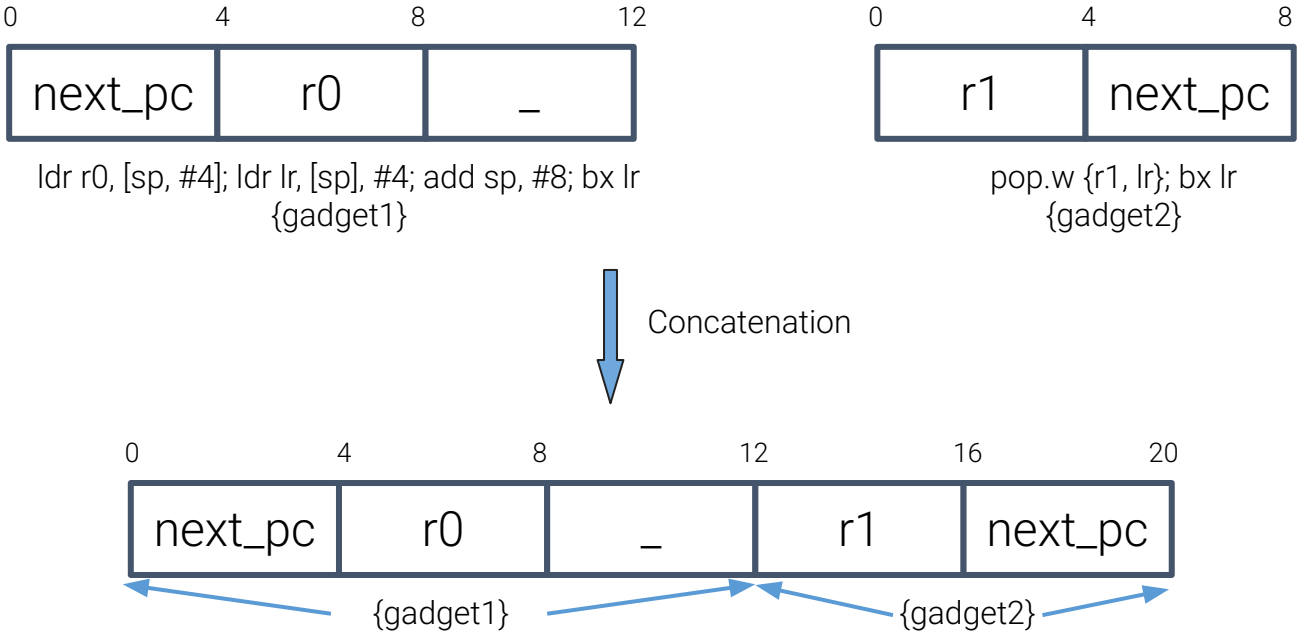


ldr r0, [sp, #4]; ldr lr, [sp], #4; add sp, #8; bx lr
{gadget1}

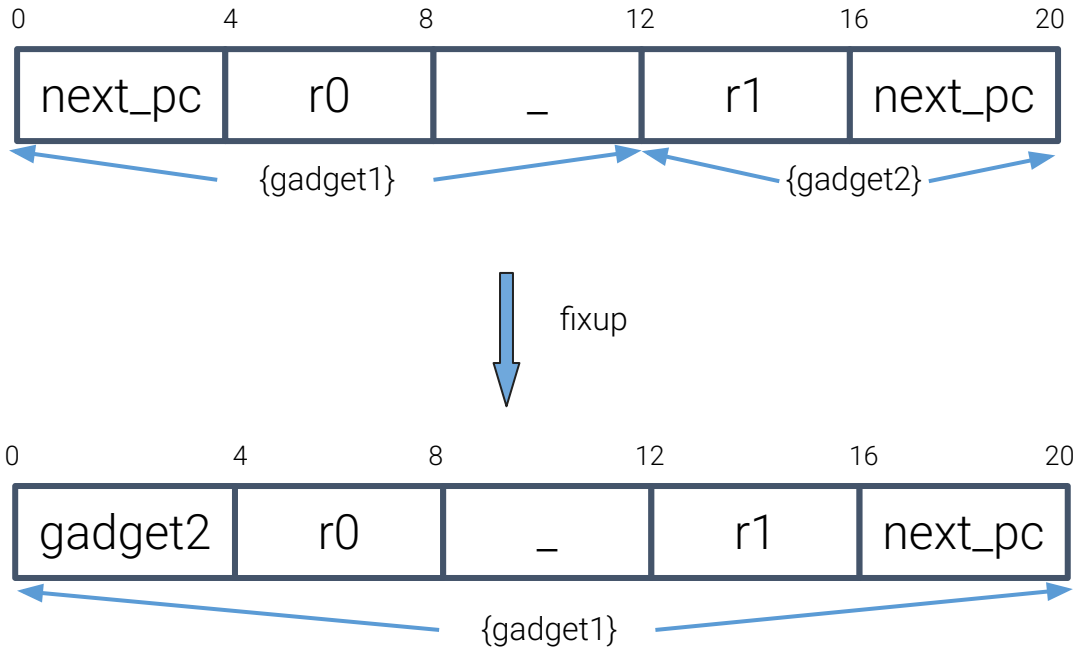


pop.w {r1, lr}; bx lr
{gadget2}

ROP Block Chaining



ROP Block Chaining

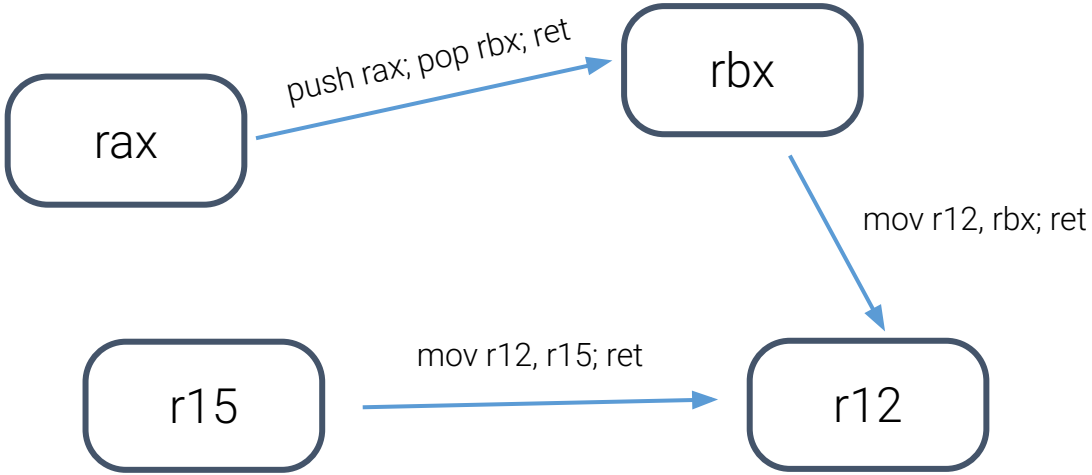


Normalization

- Goal: turn non self-contained gadgets into rop block
  a. Set branch guard for gadgets with conditional branches
  b. Do memory write for `call [mem]` gadgets
  c. Set registers for `jmp/call <reg>` gadgets

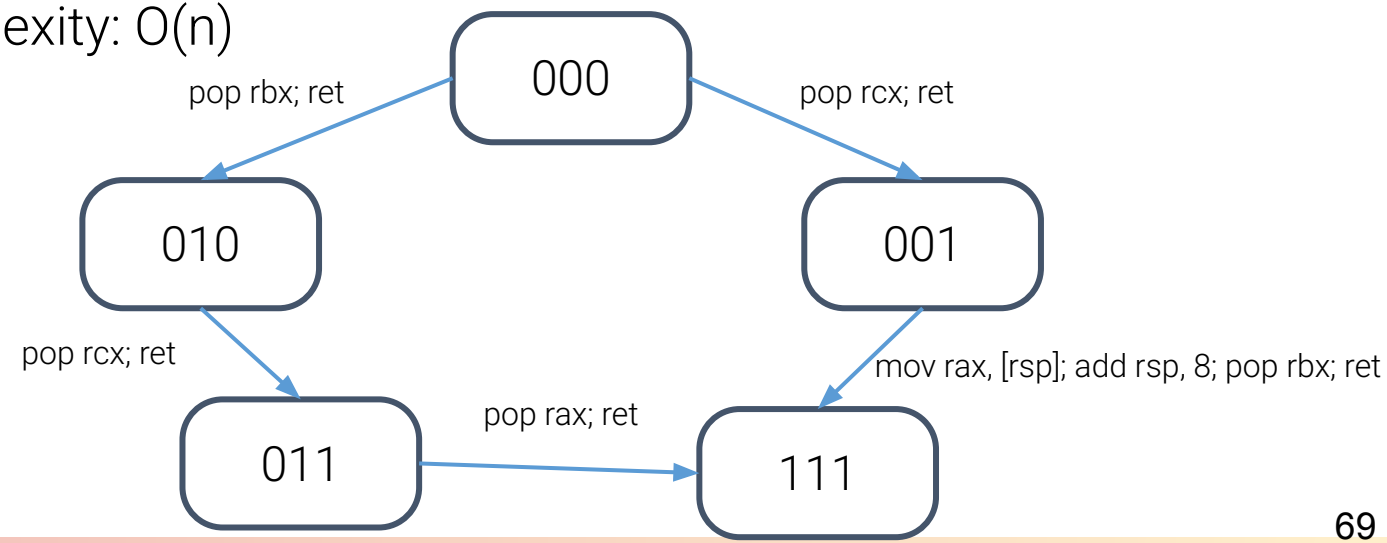
Register Moving Graph

- Capture the register moving relationship.
- New reg setting capability if 1) there is path and 2) we can set the source



Graph Search

- Given a request {rax=..., rbx=..., rcx=...}, goal: a path from 000 to 111
- 001 means, can't control {rax, rbx} and can control {rcx}
- Time Complexity: O(n)



Preliminary Result – execve(/bin/sh, 0, 0) (272 binaries)

	Success	False Positive	Time
ropbot	**244**	**0%**	**3.2h**
SGC	27	1.6%	123.h
exrop	90	23.1%	20.9h
crackers	0	N/A	55.2h

Preliminary Result – full chain (37 binaries)

full chain: dup+dup+execve

	Success	False Positive	Time
ropbot	**37**	**0%**	**1.0h**
exrop	21	4.5%	2.5h

Preliminary Result – ropbot's Cross Arch Eval

Func call: 0xfacefeed(0xdeadbeef, 0x40, 0x7b)

	Success	Total	Rate	FP	Time
x64	635	1022	62.1%	0%	5.3h
mips	285	1000	28.5%	0%	10.1h
aarch64	58	172	33.7%	0%	3.9h
arm	660	1000	66.0%	0%	4.1h
risc-v	358	1000	35.8%	0%	24.1h

ropbot Takeaways

- Identified chainability as the dominating factor
- Generalize the concept of gadget into rop_block
- New algorithm that reduce time complexity from O(n^N) to O(n)
- Outperforms all state-of-the-art works in all benchmarks

Agenda

- Background
- Exploit Generalization
- Case 1: Playing For K(H)eaps
- Case 2: RetSpill
- Case 3: ropbot
- Conclusion

Conclusion

- Proposed exploit generalization
- Successfully applied it to three different domains
- Gained deeper insights into existing systems by learning from exploits
- Derived multiple new exploitation techniques

Acknowledgement

Thank you!

Q & A

Kyle Zeng

zengyhkyle@asu.edu

@ky1ebot

@Kyle-Kyle